The Mathematics of Backpropagation: Companion Derivations for `llm.mojo`

John W. Owen III

***Abstract*— This document presents working derivations of the backward pass for every operation in the GPT-2 [2] forward pass, as implemented in the `llm.mojo` kernels. The project is directly inspired by Andrej Karpathy's `llm.c` [1]. We order the sections by backward-derivation pedagogy, starting from the scalar loss and working toward the inputs, rather than by the forward execution order of `llm.c`. This structure ensures that we derive each gradient only after deriving the gradient on which it depends. Each section anchors on the forward computation and provides a structured workspace for the derivation. The companion source code is available at https://github.com/ulmentflam/llm.mojo.**

CONTENTS

TABLE I

SYMBOLS AND THEIR CORRESPONDING KERNEL PARAMETERS.

Symbol	Code	Meaning
B	`batch_size`	batch size
T	`seq_len`	sequence length
C	`channels`	model width (embed dim)
V	`vocab_size`	real vocabulary size
V_p	`vocab_size_padded`	padded vocab (stride)
H	`num_heads`	attention heads, head $h = C/H$

I. NOTATION AND CONVENTIONS

Tensor shapes in these derivations follow the parameters of the implementation, which Table I summarizes. We define the index conventions as b for the batch, t for sequence positions, and i, j, k for channels or vocabulary entries. A row of activations at a single sequence position is denoted by the vector $x_{bt} \in \mathbb{R}^C$.

a) Why index form?: Matrix calculus obscures backpropagation under transposes and Kronecker products. This document writes every relation in *index form* (element-by-element), reducing tensor operations to scalar calculus. Treating entries as independent scalar variables allows standard derivative rules. Matching indices then reassembles scalar equations back into matrix form.

b) Linear algebra in index form.: A matrix entry is named (row, column): A_{ij} is row i, column j. Three definitions cover every matrix manipulation in this document. The transpose swaps index roles:

$$(A^\top)_{ij} \;=\; A_{ji}. \tag{1}$$

The matrix product contracts the inner indices:

$$(AB)_{ik} \;=\; \sum_j A_{ij}\, B_{jk}. \tag{2}$$

In code, this summation represents a loop over the shared dimension j:

```
val = 0.0
for j in range(width):
    val += A[i, j] * B[j, k]
```

Reversing (2) identifies matrix products. Summing over a shared index yields a product when the index is the left factor's column and the right factor's row. Elsewhere, a transpose (1) restores this alignment. This generates every transpose in the backward pass. Adding a vector to a matrix broadcasts the vector over rows:

$$(M + b)_{ni} \;=\; M_{ni} + b_i. \tag{3}$$

c) Target and probe indices.: Differentiating a tensor expression requires distinct names for the output (*target*) indices and the differentiation (*probe*) indices. Perturbing X_{mk} to find the change in Y_{ni} requires independent letters $(m, k \neq n, i)$. Reusing indices would restrict the perturbation to matching coordinate pairs. Independent probe indices evaluate the derivative of any output coordinate with respect to any input coordinate.

d) Indexed variables under differentiation.: Entries of distinct tensors are unrelated; their cross-derivatives vanish. The derivative of a tensor entry with respect to another is one for identical coordinates and zero otherwise. The Kronecker delta records this relation:

$$\delta_{ij} \;=\; \begin{cases} 1 & i = j, \\ 0 & i \neq j. \end{cases} \tag{4}$$

For a vector, one index pair determines equality; for a matrix, both row and column must match. A product of deltas encodes this conjunction:

$$\frac{\partial x_j}{\partial x_i} \;=\; \delta_{ji}, \qquad \frac{\partial W_{ij}}{\partial W_{kl}} \;=\; \delta_{ik}\,\delta_{jl}. \tag{5}$$

Two concrete matrix instances:

$$\frac{\partial W_{12}}{\partial W_{12}} = \delta_{11}\,\delta_{22} = 1 \cdot 1 = 1, \quad \frac{\partial W_{12}}{\partial W_{21}} = \delta_{12}\,\delta_{21} = 0 \cdot 0 = 0. \tag{6}$$

Deltas exit derivations by collapsing sums. For fixed l, the Kronecker delta sifts the summation:

$$\sum_j \delta_{jl}\, A_j \;=\; A_l. \tag{7}$$

This sifting acts as a conditional check inside a loop:

```
sum_val = 0.0
for j in range(N):
    if j == l:
        sum_val += 1.0 * A[j]
    else:
        sum_val += 0.0 * A[j]
# sum_val is now exactly A[l]
```

Unsummed deltas remain in the final expression, enforcing index equality.

e) The backward convention.: Each operation receives the upstream gradient $\frac{\partial L}{\partial y}$ of the scalar loss L and computes the downstream gradient $\frac{\partial L}{\partial x}$ (or parameter gradient $\frac{\partial L}{\partial w}$) via the chain rule:

$$\frac{\partial L}{\partial x_i} \;=\; \sum_j \frac{\partial L}{\partial y_j}\,\frac{\partial y_j}{\partial x_i}. \tag{8}$$

This sum accumulates gradients from all output coordinates y_j that depend on x_i. Shared inputs or parameters (such as tied weights or broadcast biases) accumulate gradients across all active positions via in-place addition (+=).

f) Layout and transposes.: The *numerator layout* convention matches the shape of the gradient $\frac{\partial L}{\partial X}$ to the source tensor X. Index-level rearrangements then generate transposes automatically, avoiding memorized matrix identities.

g) Reading order.: While the forward pass runs encoder → layernorm → attention → matmul → GELU → softmax → cross-entropy, these derivations follow the backward flow. Starting at the loss (§III) and ending at the embeddings (§IX), each section consumes the upstream gradient from the previous section. §X reassembles the complete backward pass.

II. Scalar Example: Chain Rule on a Tiny Network

We introduce a two-step composition to fix conventions before the tensor operations: $z = wx + b$, $a = \sigma(z)$, and $L = \frac{1}{2}(a - y)^2$.

Derivation. We propagate the gradient backward through the loss, then the sigmoid:

$$\frac{\partial L}{\partial a} = a - y \tag{9}$$

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \tag{10}$$

$$\frac{\partial a}{\partial z} = \sigma(z) \cdot (1 - \sigma(z)) \tag{11}$$

Because $a = \sigma(z)$,

$$\frac{\partial L}{\partial z} = (a - y) \cdot \frac{\partial a}{\partial z} = (a - y) \cdot a \cdot (1 - a). \tag{12}$$

We compute the gradient for the weight:

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial w} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial w} \tag{13}$$

$$\frac{\partial z}{\partial w} = x \tag{14}$$

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} \cdot x = (a - y) \cdot a \cdot (1 - a) \cdot x. \tag{15}$$

We compute the gradient for the bias:

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial b} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial b} \tag{16}$$

$$\frac{\partial z}{\partial b} = 1 \tag{17}$$

$$\frac{\partial L}{\partial b} = \frac{\partial L}{\partial z} = (a - y) \cdot a \cdot (1 - a). \tag{18}$$

We compute the gradient for the input:

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial a} \cdot \frac{\partial a}{\partial z} \cdot \frac{\partial z}{\partial x} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial x} \tag{19}$$

$$\frac{\partial z}{\partial x} = w \tag{20}$$

$$\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \cdot w = (a - y) \cdot a \cdot (1 - a) \cdot w. \tag{21}$$

Result. The gradient propagates in a single chain back through the network, fanning out at the affine inputs:

$$\frac{\partial L}{\partial a} \;\to\; \frac{\partial L}{\partial z} \;\to\; \left\{ \frac{\partial L}{\partial w}, \frac{\partial L}{\partial b}, \frac{\partial L}{\partial x} \right\} \tag{22}$$

where we compute and reuse $\frac{\partial L}{\partial z} = (a - y) \cdot a \cdot (1 - a)$: $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z} \cdot x$, $\frac{\partial L}{\partial b} = \frac{\partial L}{\partial z}$, $\frac{\partial L}{\partial x} = \frac{\partial L}{\partial z} \cdot w$.

III. Cross-Entropy Loss

We define the standard cross-entropy between a target distribution $y_{bt} \in \mathbb{R}^V$ and predicted probabilities $p_{bt} \in \mathbb{R}^{V_p}$ as:

$$L_{bt} \;=\; -\sum_{i<V} y_{bt,i} \log p_{bt,i}. \tag{23}$$

For language modeling, the target distribution is one-hot. Writing $\mathbb{1}[P]$ for the indicator of a condition P:

$$\mathbb{1}[P] \;=\; \begin{cases} 1 & P \text{ is true,} \\ 0 & P \text{ is false,} \end{cases} \tag{24}$$

(where the Kronecker delta (4) is this indicator specialized to index equality, $\delta_{ij} = \mathbb{1}[i = j]$), the target with token $y_{bt} \in \{0, \ldots, V-1\}$ is $y_{bt,i} = \mathbb{1}[i = y_{bt}]$, placing all probability mass on the target token and zero elsewhere. Every term of (23) except one vanishes, collapsing the loss to the form that `crossentropy_ohe_fwd` computes:

$$L_{bt} \;=\; -\log p_{bt,\,y_{bt}}. \tag{25}$$

A. Backward w.r.t. the probabilities

Derivation. We assign index i to the probed coordinate of p_{bt} and index j to the summed coordinate. Because rows are independent, the coordinates of p_{bt} within a row act as independent variables obeying (5). We expand the standard form (23) term by term:

$$\begin{aligned} L_{bt} = -\big(&y_{bt,0} \log p_{bt,0} + y_{bt,1} \log p_{bt,1} \\ &+ y_{bt,2} \log p_{bt,2} + \cdots + y_{bt,V-1} \log p_{bt,V-1}\big). \end{aligned} \tag{26}$$

When we differentiate with respect to one coordinate $p_{bt,i}$, every term of the expanded sum except the i-th term is constant, so only its derivative survives:

$$\begin{aligned} \frac{\partial L}{\partial p_{bt,i}} &= -\frac{\partial}{\partial p_{bt,i}}\big(y_{bt,0} \log p_{bt,0} + \cdots + y_{bt,i} \log p_{bt,i} \\ &\qquad + \cdots + y_{bt,V-1} \log p_{bt,V-1}\big) \\ &= -\,y_{bt,i}\,\frac{\partial}{\partial p_{bt,i}} \log p_{bt,i}. \end{aligned} \tag{27}$$

The surviving factor is the derivative of the logarithm:

$$\frac{\mathrm{d}}{\mathrm{d}x} \log x \;=\; \frac{1}{x} \qquad\Longrightarrow\qquad \frac{\partial}{\partial p_{bt,i}} \log p_{bt,i} \;=\; \frac{1}{p_{bt,i}}. \tag{28}$$

In delta form, we apply (5) followed by the sifting property (7): each term of the sum differentiates to $(y_{bt,j}/p_{bt,j})\,\delta_{ji}$, which collapses the sum to the single term where $j = i$:

$$\frac{\partial L}{\partial p_{bt,i}} \;=\; \sum_{j<V} -\frac{y_{bt,j}}{p_{bt,j}}\,\delta_{ji} \;=\; -\frac{y_{bt,i}}{p_{bt,i}}, \tag{29}$$

yielding the standard-sum gradient, which remains valid for *any* target distribution y_{bt}:

$$\frac{\partial L}{\partial p_{bt,i}} \;=\; -\,\frac{\partial}{\partial p_{bt,i}} \sum_{j<V} y_{bt,j} \log p_{bt,j} \;=\; -\frac{y_{bt,i}}{p_{bt,i}}. \tag{30}$$

When we substitute the one-hot target $y_{bt,i} = \mathbb{1}[i = y_{bt}]$, the loss (25) depends only on the target coordinate $p_{bt,\,y_{bt}}$:

$$\frac{\partial L}{\partial p_{bt,i}} = \begin{cases} -\dfrac{1}{p_{bt,\,y_{bt}}} & i = y_{bt}, \\ 0 & \text{otherwise.} \end{cases} \tag{31}$$

Result.

$$\frac{\partial L}{\partial p_{bt,i}} \;=\; -\frac{\mathbb{1}[i = y_{bt}]}{p_{bt,i}}. \tag{32}$$

IV. Softmax

We express the forward pass from logits $\ell_{bt} \in \mathbb{R}^{V_p}$ with real entries $i < V$ as:

$$p_{bt,i} \;=\; \frac{e^{\ell_{bt,i} - m_{bt}}}{\sum_{j<V} e^{\ell_{bt,j} - m_{bt}}}, \qquad m_{bt} = \max_{j<V} \ell_{bt,j}. \tag{33}$$

A. The Jacobian

We drop the row indices bt throughout this section because each row represents an independent softmax, making the Jacobian per-row.

Derivation. We seek the Jacobian entry $\frac{\partial p_i}{\partial \ell_j}$, which describes how one probability changes with respect to one logit. We assign index i to the probed probability and index j to the probed logit. The logits within a row are independent variables obeying (5), $\frac{\partial \ell_a}{\partial \ell_j} = \delta_{aj}$. The loss enters only in the subsequent sections, where we chain an upstream gradient through these entries. Before differentiating, we note that the max-subtraction in (33) cancels because for any constant c:

$$\frac{e^{\ell_i - c}}{\sum_{j<V} e^{\ell_j - c}} \;=\; \frac{e^{-c}\, e^{\ell_i}}{e^{-c} \sum_{j<V} e^{\ell_j}} \;=\; \frac{e^{\ell_i}}{\sum_{j<V} e^{\ell_j}}, \tag{34}$$

allowing us to differentiate the unshifted form. We expand the denominator sum in the same format:

$$p_i = \frac{e^{\ell_i}}{S}, \qquad S = e^{\ell_0} + e^{\ell_1} + e^{\ell_2} + \cdots + e^{\ell_{V-1}}. \tag{35}$$

We compute each entry using the quotient rule for a fraction u/v with scalar argument x:

$$\frac{\partial}{\partial x}\,\frac{u}{v} \;=\; \frac{\frac{\partial u}{\partial x}\, v \;-\; u\, \frac{\partial v}{\partial x}}{v^2}. \tag{36}$$

We calculate the two-by-two block of entries for p_1, p_2 and ℓ_1, ℓ_2 explicitly before generalizing.

a) Entry $\frac{\partial p_1}{\partial \ell_1}$: In this case, $u = e^{\ell_1}$ and $v = S$, and both variables vary with ℓ_1:

$$\frac{\partial u}{\partial \ell_1} = \frac{\partial}{\partial \ell_1}\, e^{\ell_1} = e^{\ell_1}, \tag{37}$$

$$\frac{\partial v}{\partial \ell_1} = \frac{\partial}{\partial \ell_1}\big(e^{\ell_0} + e^{\ell_1} + e^{\ell_2} + \cdots + e^{\ell_{V-1}}\big) = e^{\ell_1}, \tag{38}$$

where every term of the expanded sum except e^{ℓ_1} is constant. Substituting (37) and (38) into the quotient rule (36) yields:

$$\begin{aligned} \frac{\partial p_1}{\partial \ell_1} &= \frac{e^{\ell_1}\big(e^{\ell_0} + e^{\ell_1} + \cdots + e^{\ell_{V-1}}\big) - e^{\ell_1}\, e^{\ell_1}}{\big(e^{\ell_0} + e^{\ell_1} + \cdots + e^{\ell_{V-1}}\big)^2} \\ &= \frac{e^{\ell_1}\, S - e^{\ell_1}\, e^{\ell_1}}{S^2} = \frac{e^{\ell_1}}{S} - \left(\frac{e^{\ell_1}}{S}\right)^2 \\ &= p_1 - p_1^2 \;=\; p_1\,(1 - p_1). \end{aligned} \tag{39}$$

b) Entry $\frac{\partial p_1}{\partial \ell_2}$: Here, the numerator does not change: $u = e^{\ell_1}$ is constant with respect to ℓ_2, which gives $\frac{\partial u}{\partial \ell_2} = 0$. Again, only one term of the expanded denominator sum survives:

$$\frac{\partial v}{\partial \ell_2} = \frac{\partial}{\partial \ell_2}\left(e^{\ell_0} + e^{\ell_1} + e^{\ell_2} + \cdots + e^{\ell_{V-1}}\right) = e^{\ell_2}. \quad (40)$$

Substituting these values into (36) yields:

$$\begin{aligned}\frac{\partial p_1}{\partial \ell_2} &= \frac{0 \cdot \left(e^{\ell_0} + e^{\ell_1} + \cdots + e^{\ell_{V-1}}\right) - e^{\ell_1}\, e^{\ell_2}}{\left(e^{\ell_0} + e^{\ell_1} + \cdots + e^{\ell_{V-1}}\right)^2} \\ &= -\frac{e^{\ell_1}}{S} \cdot \frac{e^{\ell_2}}{S} \;=\; -\,p_1\, p_2. \end{aligned} \quad (41)$$

c) Entries $\frac{\partial p_2}{\partial \ell_1}$ and $\frac{\partial p_2}{\partial \ell_2}$: We perform the same two computations with the indices swapped. For $\frac{\partial p_2}{\partial \ell_1}$, $u = e^{\ell_2}$ is constant with respect to ℓ_1, and $\frac{\partial v}{\partial \ell_1} = e^{\ell_1}$ by (38). For $\frac{\partial p_2}{\partial \ell_2}$, both the numerator and the denominator vary, matching (39):

$$\frac{\partial p_2}{\partial \ell_1} = \frac{0 \cdot S - e^{\ell_2}\, e^{\ell_1}}{S^2} = -\,p_2\, p_1, \quad (42)$$

$$\frac{\partial p_2}{\partial \ell_2} = \frac{e^{\ell_2}\, S - e^{\ell_2}\, e^{\ell_2}}{S^2} = p_2\,(1 - p_2). \quad (43)$$

d) The pattern: Arranging these four entries as a block of the Jacobian yields:

$$\begin{pmatrix} \frac{\partial p_1}{\partial \ell_1} & \frac{\partial p_1}{\partial \ell_2} \\ \frac{\partial p_2}{\partial \ell_1} & \frac{\partial p_2}{\partial \ell_2} \end{pmatrix} = \begin{pmatrix} p_1\,(1 - p_1) & -\,p_1\, p_2 \\ -\,p_2\, p_1 & p_2\,(1 - p_2) \end{pmatrix}: \quad (44)$$

this block contains diagonal entries $p_i\,(1 - p_i)$ and off-diagonal entries $-\,p_i\, p_j$. The derivation of the general entry follows the same quotient rule. Each derivative of v above illustrates the general fact that exactly one term of the expanded sum S depends on ℓ_j. In delta form, we differentiate each term using (5) and collapse the sum via sifting (7):

$$\frac{\partial S}{\partial \ell_j} = \sum_{a<V} \frac{\partial}{\partial \ell_j}\, e^{\ell_a} = \sum_{a<V} e^{\ell_a}\, \delta_{aj} = e^{\ell_j}. \quad (45)$$

The diagonal case $j = i$ repeats (39) because the numerator also varies:

$$\begin{aligned}\frac{\partial p_i}{\partial \ell_i} &= \frac{e^{\ell_i}\, S - e^{\ell_i}\, e^{\ell_i}}{S^2} = \frac{e^{\ell_i}}{S} - \left(\frac{e^{\ell_i}}{S}\right)^2 \\ &= p_i - p_i^2 = p_i\,(1 - p_i). \end{aligned} \quad (46)$$

The off-diagonal case $j \neq i$ repeats (41) because the numerator e^{ℓ_i} is constant with respect to ℓ_j:

$$\frac{\partial p_i}{\partial \ell_j} = \frac{0 \cdot S - e^{\ell_i}\, e^{\ell_j}}{S^2} = -\,p_i\, p_j. \quad (47)$$

Result. The two cases combine into one expression via the Kronecker delta (4), which selects the diagonal case:

$$\frac{\partial p_i}{\partial \ell_j} \;=\; p_i\left(\delta_{ij} - p_j\right). \quad (48)$$

Check against (46) and (47): $j = i$ gives $p_i\,(1 - p_i)$, and $j \neq i$ gives $-\,p_i\, p_j$.

B. Backward w.r.t. the logits

Derivation. We apply the upstream gradient $\frac{\partial L}{\partial p}$ through the chain rule (8) summed over the row, using one column of the Jacobian (48) for each output i. The summation index j runs over the upstream coordinates, while the free index i denotes the logit gradient entry. Expanding this sum yields:

$$\begin{aligned}\frac{\partial L}{\partial \ell_i} &= \sum_{j<V} \frac{\partial L}{\partial p_j}\, \frac{\partial p_j}{\partial \ell_i} \\ &= \frac{\partial L}{\partial p_0}\, \frac{\partial p_0}{\partial \ell_i} + \frac{\partial L}{\partial p_1}\, \frac{\partial p_1}{\partial \ell_i} + \frac{\partial L}{\partial p_2}\, \frac{\partial p_2}{\partial \ell_i} \\ &\quad + \cdots + \frac{\partial L}{\partial p_{V-1}}\, \frac{\partial p_{V-1}}{\partial \ell_i}. \end{aligned} \quad (49)$$

We substitute (46) and (47), separating the diagonal term $j = i$ from the off-diagonal terms:

$$\begin{aligned}\frac{\partial L}{\partial \ell_i} &= \frac{\partial L}{\partial p_i}\, p_i\,(1 - p_i) + \sum_{j \neq i} \frac{\partial L}{\partial p_j}\,(-\,p_j\, p_i) \\ &= p_i\, \frac{\partial L}{\partial p_i} - p_i\, p_i\, \frac{\partial L}{\partial p_i} - p_i \sum_{j \neq i} \frac{\partial L}{\partial p_j}\, p_j. \end{aligned} \quad (50)$$

Because the middle term corresponds exactly to the missing $j = i$ term of the sum, we absorb it back into the summation:

$$\frac{\partial L}{\partial \ell_i} = p_i\, \frac{\partial L}{\partial p_i} - p_i \sum_{j<V} \frac{\partial L}{\partial p_j}\, p_j, \quad (51)$$

where the remaining sum is identical for every i in the row. Expanding this term yields:

$$\begin{aligned}\sum_{j<V} \frac{\partial L}{\partial p_j}\, p_j &= \frac{\partial L}{\partial p_0}\, p_0 + \frac{\partial L}{\partial p_1}\, p_1 \\ &\quad + \frac{\partial L}{\partial p_2}\, p_2 + \cdots + \frac{\partial L}{\partial p_{V-1}}\, p_{V-1}. \end{aligned} \quad (52)$$

Result.

$$\frac{\partial L}{\partial \ell_i} \;=\; p_i \left(\frac{\partial L}{\partial p_i} - \sum_{j<V} \frac{\partial L}{\partial p_j}\, p_j\right), \qquad i < V, \quad (53)$$

and $\frac{\partial L}{\partial \ell_i} = 0$ for the padded region $i \geq V$. When we sum (53) over the row, the leading factor p_i distributes into both terms. The probabilities multiplying the constant dot product sum to one, leaving two copies of the dot product:

$$\begin{aligned}\sum_{i<V} \frac{\partial L}{\partial \ell_i} &= \sum_{i<V} p_i\, \frac{\partial L}{\partial p_i} - \left(\sum_{i<V} p_i\right) \sum_{j<V} \frac{\partial L}{\partial p_j}\, p_j \\ &= \sum_{i<V} p_i\, \frac{\partial L}{\partial p_i} - \sum_{j<V} \frac{\partial L}{\partial p_j}\, p_j \;=\; 0. \end{aligned} \quad (54)$$

The gradient row sums to zero, which reflects the shift invariance (34).

C. Fused softmax + cross-entropy

This composition represents the primary use case for the model. Backpropagating through (25) and (33) simultaneously collapses the Jacobian.

Derivation. We write the chain rule per logit coordinate, restoring the full indices. The summation index i runs over the probabilities, while the free index j denotes the logit gradient entry:

$$\frac{\partial L}{\partial \ell_{bt,j}} = \sum_{i<V} \frac{\partial L}{\partial p_{bt,i}} \frac{\partial p_{bt,i}}{\partial \ell_{bt,j}}. \tag{55}$$

We substitute the cross-entropy gradient (32) for the first factor and the Jacobian (48) for the second factor into the combined equation:

$$\frac{\partial L}{\partial \ell_{bt,j}} = \sum_{i<V} \left(-\frac{\mathbb{1}[i = y_{bt}]}{p_{bt,i}} \right) p_{bt,i} \left(\delta_{ij} - p_{bt,j} \right) \tag{56}$$

The factor $p_{bt,i}$ in the denominator cancels with the factor $p_{bt,i}$ from the Jacobian:

$$= -\sum_{i<V} \mathbb{1}[i = y_{bt}] \left(\delta_{ij} - p_{bt,j} \right) \tag{57}$$

and the one-hot indicator isolates the single term where $i = y_{bt}$:

$$= -\left(\delta_{y_{bt}j} - p_{bt,j} \right) = p_{bt,j} - \mathbb{1}[j = y_{bt}]. \tag{58}$$

Result.

$$\frac{\partial L}{\partial \ell_{bt,j}} = p_{bt,j} - \mathbb{1}[j = y_{bt}], \qquad j < V, \tag{59}$$

and $\frac{\partial L}{\partial \ell_{bt,j}} = 0$ for the padded region $j \geq V$. The Jacobian collapses entirely, removing the sum over the row in (59).

We verify this result via the general backward formula (53). With the cross-entropy upstream gradient (32), the dot product evaluates to:

$$\sum_{j<V} \frac{\partial L}{\partial p_{bt,j}} p_{bt,j} = \sum_{j<V} \left(-\frac{\mathbb{1}[j = y_{bt}]}{p_{bt,j}} \right) p_{bt,j} = -1, \tag{60}$$

which is a constant. For a one-hot target, we determine the row-wide weighted sum before reading any element. Substituting -1 for the dot product in (53), the factor $p_{bt,i}$ outside the parentheses cancels the reciprocal inside:

$$\begin{aligned} \frac{\partial L}{\partial \ell_{bt,i}} &= p_{bt,i} \left(\frac{\partial L}{\partial p_{bt,i}} - (-1) \right) \\ &= p_{bt,i} \left(-\frac{\mathbb{1}[i = y_{bt}]}{p_{bt,i}} + 1 \right) \\ &= p_{bt,i} - \mathbb{1}[i = y_{bt}], \end{aligned} \tag{61}$$

demonstrating that chaining through the Jacobian directly and substituting into the general backward formula represent two equivalent routes to the same gradient. The zero-sum property of §IV also holds because the probabilities sum to one, and the indicator fires exactly once per row:

$$\sum_{j<V} \left(p_{bt,j} - \mathbb{1}[j = y_{bt}] \right) = 1 - 1 = 0. \tag{62}$$

The collapse does not require a one-hot target. It requires only the standard-sum gradient (30) and the assumption that the target is a probability distribution satisfying $\sum_{i<V} y_{bt,i} = 1$. Repeating (55) with a general target distribution:

$$\begin{aligned} \frac{\partial L}{\partial \ell_{bt,j}} &= \sum_{i<V} \left(-\frac{y_{bt,i}}{p_{bt,i}} \right) p_{bt,i} \left(\delta_{ij} - p_{bt,j} \right) \\ &= -y_{bt,j} + p_{bt,j} \sum_{i<V} y_{bt,i} = p_{bt,j} - y_{bt,j}, \end{aligned} \tag{63}$$

where the one-hot result (59) is the special case where $y_{bt,j} = \mathbb{1}[j = y_{bt}]$. Thus, softmax paired with cross-entropy always backpropagates as *predicted minus target*.

V. Linear Layer (Matmul)

We express the forward pass as $Y = XW^\top + b$ with $X \in \mathbb{R}^{(B \cdot T) \times C_{\text{in}}}$, $W \in \mathbb{R}^{C_{\text{out}} \times C_{\text{in}}}$, and $b \in \mathbb{R}^{C_{\text{out}}}$.

Derivation. We write the equations in index form: n runs over the $N = B \cdot T$ flattened rows ($n = bt$), j runs over C_{in}, and i runs over C_{out}. We interpret every matrix entry as (row, column): X_{nj} is (position n, in-channel j); W_{ij} is (out-channel i, in-channel j); Y_{ni} is (position n, out-channel i); and b_i has the single index (out-channel i).

Differentiation requires a second, independent set of index names. To determine how Y_{ni} changes when one entry of X varies, we must allow the perturbed entry to be any element. Thus, the probe entry receives fresh indices: X_{mk} (position m, in-channel k), and similarly W_{kl} (out-channel k, in-channel l) and b_k. Keeping the target indices (n, i) and the probe indices distinct evaluates every combination of output and input simultaneously.

a) The transpose under differentiation: By the transpose definition (1), we write $(W^\top)_{ji} = W_{ij}$, which relabels the same independent variables. Applying (5) through the relabeled indices, we find that the transpose changes only which index of W we compare with the probe index:

$$\frac{\partial (W^\top)_{ji}}{\partial W_{kl}} = \frac{\partial W_{ij}}{\partial W_{kl}} = \delta_{ik}\, \delta_{jl}. \tag{64}$$

After absorbing (1) and writing out the broadcast bias (3), we express the forward pass in index form as:

$$\begin{aligned} Y_{ni} &= \sum_{j<C_{\text{in}}} X_{nj} \, (W^\top)_{ji} + b_i = \sum_{j<C_{\text{in}}} X_{nj} \, W_{ij} + b_i \\ &= X_{n0} \, W_{i0} + X_{n1} \, W_{i1} + \cdots + X_{n,C_{\text{in}}-1} \, W_{i,C_{\text{in}}-1} + b_i. \end{aligned} \tag{65}$$

b) Local derivative $\frac{\partial Y}{\partial X}$: With target Y_{ni} (position n, out-channel i) and probe X_{mk} (position m, in-channel k), we consider four free indices, representing every pair of output and input entries. We differentiate the expanded form (65) term by term. Because W and b act as constants, and each X_{nj} obeys (5), we obtain:

$$\begin{aligned} \frac{\partial Y_{ni}}{\partial X_{mk}} &= \sum_{j<C_{\text{in}}} W_{ij} \frac{\partial X_{nj}}{\partial X_{mk}} = \sum_{j<C_{\text{in}}} W_{ij} \, \delta_{nm} \, \delta_{jk} \\ &= \delta_{nm} \sum_{j<C_{\text{in}}} W_{ij} \, \delta_{jk} = \delta_{nm} \, W_{ik}. \end{aligned} \tag{66}$$

Because the row delta δ_{nm} does not depend on j, we factor it out of the sum. The column delta sifts the sum to $j = k$ via (7). The delta δ_{nm} survives because we do not sum over n or m here. This term remains for the chain rule, indicating that row n of Y depends only on row $m = n$ of X.

c) Local derivative $\frac{\partial Y}{\partial W}$: With target Y_{ni} and probe W_{kl} (out-channel k, in-channel l), X acts as the constant and each W_{ij} obeys (64):

$$\begin{aligned}\frac{\partial Y_{ni}}{\partial W_{kl}} &= \sum_{j<C_{\text{in}}} X_{nj}\,\frac{\partial W_{ij}}{\partial W_{kl}} = \sum_{j<C_{\text{in}}} X_{nj}\,\delta_{ik}\,\delta_{jl} \\ &= \delta_{ik}\sum_{j<C_{\text{in}}} X_{nj}\,\delta_{jl} = \delta_{ik}\,X_{nl}. \end{aligned} \tag{67}$$

This calculation mirrors the previous derivation with the roles swapped: the out-channel delta δ_{ik} does not depend on j and factors out, while the in-channel delta sifts the sum over j to l.

d) Local derivative $\frac{\partial Y}{\partial b}$ and its broadcast behavior: The bias term in (65) is b_i. Because b is a vector, (5) requires only one index pair:

$$\frac{\partial Y_{ni}}{\partial b_k} \;=\; \frac{\partial b_i}{\partial b_k} \;=\; \delta_{ik}, \tag{68}$$

which does not depend on n. This independence reflects the broadcast operation: the same b_k appears in every one of the N rows of Y, meaning that b_k affects N distinct output coordinates. While this local derivative resembles the scalar example (§II, $\frac{\partial z}{\partial b} = 1$), the broadcast behavior introduces a sum in the chain rule that we must not omit.

e) Chain rule: Because L depends on every entry of Y, the chain rule (8) sums over both output indices (n, i). The probe indices act as the free indices, specifying the gradient entry we compute:

$$\frac{\partial L}{\partial \theta} \;=\; \sum_{n<N}\sum_{i<C_{\text{out}}} \frac{\partial L}{\partial Y_{ni}}\,\frac{\partial Y_{ni}}{\partial \theta}. \tag{69}$$

For the input gradient, we substitute (66) into the chain rule. The surviving row delta sifts the sum over n to m. Because no delta constrains i, the sum over i remains as a contraction with W:

$$\begin{aligned}\frac{\partial L}{\partial X_{mk}} &= \sum_{n<N}\sum_{i<C_{\text{out}}} \frac{\partial L}{\partial Y_{ni}}\,\delta_{nm}\,W_{ik} \\ &= \sum_{i<C_{\text{out}}} \frac{\partial L}{\partial Y_{mi}}\,W_{ik} = \left(\frac{\partial L}{\partial Y}\,W\right)_{mk}. \end{aligned} \tag{70}$$

This expression has free indices (m, k) on both sides, corresponding to row m, in-channel k of the input gradient. The final step applies the recognition rule of (2): the summed index i is the column of $\frac{\partial L}{\partial Y}$ (in $\frac{\partial L}{\partial Y_{mi}}$) and the row of W (in W_{ik}), meaning the sum represents a matrix product without transposing either factor.

For the weight gradient, we substitute (67) into the chain rule. The out-channel delta sifts the sum over i to k, but because no delta constrains n, the sum over n survives:

$$\begin{aligned}\frac{\partial L}{\partial W_{kl}} &= \sum_{n<N}\sum_{i<C_{\text{out}}} \frac{\partial L}{\partial Y_{ni}}\,\delta_{ik}\,X_{nl} \\ &= \sum_{n<N} \frac{\partial L}{\partial Y_{nk}}\,X_{nl} = \left(\frac{\partial L}{\partial Y}^{\top} X\right)_{kl}. \end{aligned} \tag{71}$$

This expression has free indices (k, l), which match the out-channel k and in-channel l layout of W. Here, the recognition rule of (2) requires a transpose because the summed index n is the row of both $\frac{\partial L}{\partial Y}$ and X. Transposing the left factor yields $(\frac{\partial L}{\partial Y}^{\top})_{kn} = \frac{\partial L}{\partial Y_{nk}}$ via (1), which expresses the sum as a matrix product.

For the bias gradient, we substitute (68) into the chain rule. The delta sifts the sum over i. Because the local derivative does not depend on n, the sum over n survives. Expanding this sum yields:

$$\begin{aligned}\frac{\partial L}{\partial b_k} &= \sum_{n<N}\sum_{i<C_{\text{out}}} \frac{\partial L}{\partial Y_{ni}}\,\delta_{ik} = \sum_{n<N} \frac{\partial L}{\partial Y_{nk}} \\ &= \frac{\partial L}{\partial Y_{0k}} + \frac{\partial L}{\partial Y_{1k}} + \frac{\partial L}{\partial Y_{2k}} + \cdots + \frac{\partial L}{\partial Y_{N-1,k}}. \end{aligned} \tag{72}$$

This demonstrates the consequence of the broadcast behavior described in (68): although the local derivative is simply δ_{ik}, the gradient requires a summation over all $B \cdot T$ rows. In the scalar example (§II), the same relation holds with $N = 1$. The sum still exists there, but collapses to a single term, yielding $\frac{\partial L}{\partial b} = \frac{\partial L}{\partial z}$.

Result.

$$\frac{\partial L}{\partial X} \;=\; \frac{\partial L}{\partial Y}\,W, \tag{73}$$

$$\frac{\partial L}{\partial W} \;=\; \frac{\partial L}{\partial Y}^{\top} X, \tag{74}$$

$$\frac{\partial L}{\partial b_k} \;=\; \sum_{n<N} \frac{\partial L}{\partial Y_{nk}}. \tag{75}$$

We verify the dimensions of each gradient with $\frac{\partial L}{\partial Y} \in \mathbb{R}^{N\times C_{\text{out}}}$: $(N \times C_{\text{out}})(C_{\text{out}} \times C_{\text{in}}) = N \times C_{\text{in}}$ for $\frac{\partial L}{\partial X}$; $(C_{\text{out}} \times N)(N \times C_{\text{in}}) = C_{\text{out}} \times C_{\text{in}}$ for $\frac{\partial L}{\partial W}$; $\frac{\partial L}{\partial b} \in \mathbb{R}^{C_{\text{out}}}$.

The reduction over the $B \cdot T$ dimension occurs in $\frac{\partial L}{\partial W}$ and $\frac{\partial L}{\partial b}$ because every position contributes to the shared parameters (using the $+=$ convention of §I). In contrast, we compute a separate gradient row for each input row in $\frac{\partial L}{\partial X}$.

Because the forward pass contains the transpose ($Y = XW^{\top}$), two consequences follow. First, computing the input gradient requires no transpose ($\frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y}\,W$, rather than $\frac{\partial L}{\partial Y}\,W^{\top}$). Second, we obtain $\frac{\partial L}{\partial W}$ directly in the original layout of W ($C_{\text{out}} \times C_{\text{in}}$). Both properties derive from (64) rather than arbitrary conventions.

VI. GELU

The Gaussian Error Linear Unit (GELU) activation function, proposed by Hendrycks and Gimpel [4], scales the input by the cumulative normal distribution function $\text{GELU}(x) = x\Phi(x)$. Because the exact CDF involves the non-elementary error function, we express the forward pass using the fast

hyperbolic tangent approximation introduced by Page [5] and adopted by OpenAI in GPT-2 [2] and GPT-3:

$$\mathrm{GELU}(x) \;=\; \tfrac{1}{2}\,x\left(1+\tanh\left[\sqrt{2/\pi}\,(x+0.044715\,x^3)\right]\right). \tag{76}$$

We define the inner argument $u(x)$ to simplify the chain rule:

$$\begin{aligned} u(x) &= \sqrt{2/\pi}\,\big(x+0.044715\,x^3\big),\\ \mathrm{GELU}(x) &= \tfrac{1}{2}\,x\,\big(1+\tanh u\big). \end{aligned} \tag{77}$$

Derivation. We compute the scalar derivative $\mathrm{GELU}'(x)$. The result describes its role in the chain rule.

a) Product rule: We apply the product rule to (77):

$$\mathrm{GELU}'(x) \;=\; \tfrac{1}{2}\big(1+\tanh u\big) + \tfrac{1}{2}\,x\,\frac{\partial}{\partial x}\tanh u. \tag{78}$$

b) The tanh derivative via sech^2*:* The derivative of $\tanh$ is sech^2, which requires chaining the derivative of the inner function:

$$\frac{\partial}{\partial x}\tanh u \;=\; \mathrm{sech}^2(u)\,u'(x), \tag{79}$$

with the polynomial derivative of the inner argument:

$$\begin{aligned} u'(x) &= \sqrt{2/\pi}\,\big(1+3\cdot 0.044715\,x^2\big)\\ &= \sqrt{2/\pi}\,\big(1+0.134145\,x^2\big). \end{aligned} \tag{80}$$

c) Converting sech^2 *to* $1-\tanh^2$*:* Dividing the hyperbolic Pythagorean identity $\cosh^2 u-\sinh^2 u=1$ by $\cosh^2 u$ yields:

$$1-\tanh^2 u \;=\; \mathrm{sech}^2\,u, \tag{81}$$

so (79) needs no separate sech evaluation:

$$\frac{\partial}{\partial x}\tanh u \;=\; \big(1-\tanh^2 u\big)\,u'(x). \tag{82}$$

d) Simplifying the derivative: Because $\tanh$ differentiates into a function of itself ($\tanh' = 1-\tanh^2$), and $\tanh u$ appears in the forward expression (77), we solve for it when $x\neq 0$:

$$1+\tanh u = \frac{2\;\mathrm{GELU}(x)}{x} \;\Longrightarrow\; \tanh u = \frac{2\;\mathrm{GELU}(x)}{x}-1. \tag{83}$$

We substitute (82) into (78). This ensures that every quantity in the derivative is either the input x or the value $\tanh u$ computed during the forward pass:

$$\begin{aligned} \mathrm{GELU}'(x) &= \tfrac{1}{2}\big(1+\tanh u\big) + \tfrac{1}{2}\,x\,\big(1-\tanh^2 u\big)\,u'(x)\\ &= \frac{\mathrm{GELU}(x)}{x} + \tfrac{1}{2}\,x\,\big(1-\tanh^2 u\big)\,u'(x). \end{aligned} \tag{84}$$

Result.

$$\mathrm{GELU}'(x) \;=\; \tfrac{1}{2}\big(1+\tanh u\big) + \tfrac{1}{2}\,x\,\big(1-\tanh^2 u\big)\,u'(x), \tag{85}$$

where we compute u and u' via (77) and (80). A single evaluation of $\tanh u$ serves both terms because the self-reference $\tanh' = 1-\tanh^2$ (81) replaces sech, and (83) expresses the first term as the forward output divided by x. At $x=0$, we use (85) directly: $u(0)=0$ and $\tanh 0 = 0$ yield $\mathrm{GELU}'(0)=\frac{1}{2}$, avoiding the singularity in the $\mathrm{GELU}(x)/x$ term of (84).

GELU operates elementwise: each output coordinate depends only on its corresponding input coordinate. By (5), the Jacobian is diagonal:

$$\frac{\partial\,\mathrm{GELU}(x_a)}{\partial x_b} \;=\; \mathrm{GELU}'(x_a)\,\delta_{ab}, \tag{86}$$

and the chain rule (8) collapses the sum via sifting (7) to a pointwise product: $\frac{\partial L}{\partial x_a} = \frac{\partial L}{\partial y_a}\;\mathrm{GELU}'(x_a)$.

VII. LayerNorm

Layer normalization, proposed by Ba et al. [6], stabilizes training dynamics by normalizing activations across the channel dimension within each sequence position. We compute the forward pass at each position $x\in\mathbb{R}^C$ with scale γ and shift β as:

$$\begin{aligned} &\mu = \tfrac{1}{C}\sum_j x_j, \qquad \sigma^2 = \tfrac{1}{C}\sum_j (x_j-\mu)^2+\varepsilon,\\ &\hat{x}_i = \frac{x_i-\mu}{\sqrt{\sigma^2}},\\ &y_i = \gamma_i\,\hat{x}_i + \beta_i. \end{aligned} \tag{87}$$

Derivation. We compute the partial derivatives of the output y_i with respect to its parameters and inputs. The output y_i depends on every x_j in the row through the statistics μ and σ^2.

a) Parameter partials: The output y_i depends only on the scale and shift parameters of its corresponding channel:

$$\frac{\partial y_i}{\partial \gamma_i} = \frac{x_i-\mu}{\sqrt{\sigma^2}}, \qquad \frac{\partial y_i}{\partial \beta_i} = 1. \tag{88}$$

b) Input partials: To find the Jacobian $\frac{\partial y_j}{\partial x_i}$, we first differentiate the row statistics. Each input element x_i contributes $1/C$ to the mean and $2(x_i-\mu)/C$ to the variance:

$$\frac{\partial \mu}{\partial x_i} = \frac{1}{C}, \qquad \frac{\partial \sigma^2}{\partial x_i} = \frac{2(x_i-\mu)}{C}. \tag{89}$$

We differentiate the normalized value $\hat{x}_j$ with respect to x_i using the quotient rule on $\hat{x}_j = (x_j-\mu)(\sigma^2)^{-1/2}$:

$$\begin{aligned} \frac{\partial \hat{x}_j}{\partial x_i} &= \frac{\partial}{\partial x_i}\left(\frac{x_j-\mu}{\sqrt{\sigma^2}}\right)\\ &= \frac{(\delta_{ji}-\frac{1}{C})\sqrt{\sigma^2} - (x_j-\mu)\frac{1}{2}(\sigma^2)^{-1/2}\frac{\partial \sigma^2}{\partial x_i}}{\sigma^2}\\ &= \frac{1}{\sqrt{\sigma^2}}\left(\delta_{ji} - \frac{1}{C} - \frac{(x_j-\mu)(x_i-\mu)}{C\sigma^2}\right). \end{aligned} \tag{90}$$

The partial derivative $\frac{\partial y_j}{\partial x_i} = \gamma_j \frac{\partial \hat{x}_j}{\partial x_i}$ describes how a change in x_i affects every output in the row.

Result. We compute the total gradient for each quantity by summing its contributions to the loss across all outputs y_j, substituting the partial derivatives derived above:

$$\frac{\partial L}{\partial \gamma_i} = \frac{\partial L}{\partial y_i}\frac{\partial y_i}{\partial \gamma_i} = \frac{\partial L}{\partial y_i}\frac{x_i-\mu}{\sqrt{\sigma^2}}, \tag{91}$$

$$\frac{\partial L}{\partial \beta_i} = \frac{\partial L}{\partial y_i}\frac{\partial y_i}{\partial \beta_i} = \frac{\partial L}{\partial y_i}. \tag{92}$$

For the input gradient $\frac{\partial L}{\partial x_i}$, we sum over the entire row $j \in \{0, \ldots, C-1\}$. We expand the Kronecker delta δ_{ji} into its indicator form $\mathbb{1}[j=i]$ to clarify the sifting process:

$$\begin{aligned}
\frac{\partial L}{\partial x_i} &= \sum_j \frac{\partial L}{\partial y_j} \frac{\partial y_j}{\partial x_i} \\
&= \sum_j \frac{\partial L}{\partial y_j} \left[\frac{\gamma_j}{\sqrt{\sigma^2}} \left(\delta_{ji} - \frac{1}{C} - \frac{\hat{x}_i \hat{x}_j}{C} \right) \right] \\
&= \frac{1}{\sqrt{\sigma^2}} \left(\sum_j \gamma_j \frac{\partial L}{\partial y_j} \mathbb{1}[j=i] - \frac{1}{C} \sum_j \gamma_j \frac{\partial L}{\partial y_j} \right. \\
&\qquad \left. - \frac{\hat{x}_i}{C} \sum_j \gamma_j \frac{\partial L}{\partial y_j} \hat{x}_j \right) \\
&= \frac{1}{\sqrt{\sigma^2}} \left(\gamma_i \frac{\partial L}{\partial y_i} - \frac{1}{C} \sum_j \gamma_j \frac{\partial L}{\partial y_j} \right. \\
&\qquad \left. - \frac{\hat{x}_i}{C} \sum_j \gamma_j \frac{\partial L}{\partial y_j} \hat{x}_j \right). \qquad (93)
\end{aligned}$$

The row-wide sums $\sum \gamma_j \frac{\partial L}{\partial y_j}/C$ and $\sum \gamma_j \frac{\partial L}{\partial y_j} \hat{x}_j/C$ match the row statistics s_1 and s_2 that the first pass of `layernorm_bwd` computes.

VIII. Attention

Multi-head self-attention, the central building block of the Transformer architecture [8], allows sequence elements to dynamically route information by computing scaled dot-product attention over queries, keys, and values. We express the forward pass for each head as $A = \mathrm{softmax}\big(QK^\top/\sqrt{h} + M\big)$ and $Y = AV$, with causal mask M and projections $Q = XW_Q$, $K = XW_K$, and $V = XW_V$.

Derivation. We write the equations in index form: i runs over query positions T ($i < T$), j runs over key/value positions T ($j < T$), and d runs over the head dimension h ($d < h$). We drop the batch and head indices throughout this section because each head is independent, allowing us to derive the gradients per-head. The channel size of the query and key projections within each head is the head dimension h, corresponding to the standard notation d_k in the literature (which scales the attention scores by $1/\sqrt{d_k}$). We obtain the head dimension as $h = C/H$, where C is the total channel width (embedding dimension) and H is the number of attention heads (see Table I in §I). We write the scaling factor as $1/\sqrt{h}$ to match the head dimension symbol in the kernel code. We interpret each entry as (row, column): Q_{id} is (query position i, channel d); K_{jd} is (key position j, channel d); V_{jd} is (value position j, channel d); Y_{id} is (output position i, channel d); S_{ij} is the raw score at (query position i, key position j); and A_{ij} is the attention probability at (query position i, key position j).

The forward pass computes the raw causal score matrix $S \in \mathbb{R}^{T \times T}$ as:

$$S_{ij} \;=\; \begin{cases} \frac{1}{\sqrt{h}} \sum_{d<h} Q_{id} K_{jd} & j \le i, \\ -\infty & j > i. \end{cases} \qquad (94)$$

We obtain the attention probability matrix $A \in \mathbb{R}^{T \times T}$ by applying the row-wise softmax to S:

$$A_{ij} \;=\; \mathrm{softmax}(S_{i,\cdot})_j \;=\; \frac{e^{S_{ij} - L_i}}{\sum_{l \le i} e^{S_{il} - L_i}}, \qquad (95)$$

where $L_i = \log \sum_{l \le i} e^{S_{il}}$ is the log-sum-exp (LSE) value for row i. By definition, $A_{ij} = 0$ for $j > i$. The head output $Y \in \mathbb{R}^{T \times h}$ is the matrix product:

$$Y_{id} \;=\; \sum_{j<T} A_{ij} V_{jd} \quad \Longrightarrow \quad Y \;=\; AV. \qquad (96)$$

Differentiation requires a second, independent set of index names to avoid collisions. We choose the probe indices m for position and k for channel.

a) Local derivative $\frac{\partial Y}{\partial V}$: With target Y_{id} (query position i, channel d) and probe V_{mk} (key position m, channel k), we differentiate:

$$\begin{aligned}
\frac{\partial Y_{id}}{\partial V_{mk}} &= \frac{\partial}{\partial V_{mk}} \sum_{a<T} A_{ia} V_{ad} \\
&= \sum_{a<T} A_{ia} \delta_{am} \delta_{dk} \\
&= A_{im} \delta_{dk}. \qquad (97)
\end{aligned}$$

The Kronecker deltas act as coordinate checks: δ_{dk} restricts the channel index to $d = k$, while δ_{am} collapses the sum over position a to the single term where $a = m$. We chain the upstream gradient $\frac{\partial L}{\partial Y}$ using the multivariable chain rule (8):

$$\begin{aligned}
\frac{\partial L}{\partial V_{mk}} &= \sum_{i<T} \sum_{d<h} \frac{\partial L}{\partial Y_{id}} \frac{\partial Y_{id}}{\partial V_{mk}} \\
&= \sum_{i<T} \sum_{d<h} \frac{\partial L}{\partial Y_{id}} A_{im} \delta_{dk} \\
&= \sum_{i<T} A_{im} \frac{\partial L}{\partial Y_{ik}} \\
&= \sum_{i<T} (A^\top)_{mi} \frac{\partial L}{\partial Y_{ik}} = \left(A^\top \frac{\partial L}{\partial Y} \right)_{mk}. \qquad (98)
\end{aligned}$$

In the third step, the channel delta δ_{dk} sifts the sum over d to k. In the fourth step, we write $A_{im} = (A^\top)_{mi}$ to transpose the attention matrix; this aligns the summed index i as the inner dimension of the matrix product, matching the recognition rule of (2) to yield $\frac{\partial L}{\partial V} = A^\top \frac{\partial L}{\partial Y}$.

b) Local derivative $\frac{\partial Y}{\partial A}$: With target Y_{id} and probe A_{mj} (query position m, key position j):

$$\begin{aligned}
\frac{\partial Y_{id}}{\partial A_{mj}} &= \frac{\partial}{\partial A_{mj}} \sum_{a<T} A_{ia} V_{ad} \\
&= \sum_{a<T} \delta_{im} \delta_{aj} V_{ad} \\
&= \delta_{im} V_{jd}. \qquad (99)
\end{aligned}$$

The delta δ_{im} aligns the target query position i with the probed position m, while δ_{aj} sifts the sum over a to j. We chain the upstream gradient:

$$\begin{aligned}\frac{\partial L}{\partial A_{mj}} &= \sum_{i<T}\sum_{d<h} \frac{\partial L}{\partial Y_{id}}\frac{\partial Y_{id}}{\partial A_{mj}} \\ &= \sum_{i<T}\sum_{d<h} \frac{\partial L}{\partial Y_{id}}\,\delta_{im}\,V_{jd} \\ &= \sum_{d<h} \frac{\partial L}{\partial Y_{md}}\,V_{jd} = \left(\frac{\partial L}{\partial Y}\,V^\top\right)_{mj}. \end{aligned} \tag{100}$$

Here, the delta δ_{im} sifts the sum over i to m. The remaining sum contracts over the channel index d, representing the row of $\frac{\partial L}{\partial Y}$ and the column of $V^\top$ (via $(V^\top)_{dj} = V_{jd}$), which matches the matrix multiplication definition (2) and yields $\frac{\partial L}{\partial A} = \frac{\partial L}{\partial Y}\,V^\top$.

c) Backpropagation through row-wise Softmax: Each row of the attention matrix A is an independent softmax over the corresponding row of raw scores S. Applying the softmax gradient result (53) directly to each row i yields:

$$\frac{\partial L}{\partial S_{ij}} \;=\; A_{ij}\left(\frac{\partial L}{\partial A_{ij}} - D_i\right), \tag{101}$$

where the row-wise dot product statistics term is defined as:

$$D_i \;=\; \sum_{k<T}\frac{\partial L}{\partial A_{ik}}\,A_{ik}. \tag{102}$$

Standard backpropagation materializes the full $T\times T$ gradient matrix $\frac{\partial L}{\partial A}$ and computes the row-wise statistics D_i before evaluating $\frac{\partial L}{\partial S_{ij}}$.

d) Local derivative $\frac{\partial S}{\partial Q}$: With target score S_{ij} and probe query Q_{mk} (query position m, channel k):

$$\begin{aligned}\frac{\partial S_{ij}}{\partial Q_{mk}} &= \frac{\partial}{\partial Q_{mk}}\left(\frac{1}{\sqrt{h}}\sum_{d<h} Q_{id}\,K_{jd}\right) \\ &= \frac{1}{\sqrt{h}}\sum_{d<h}\delta_{im}\,\delta_{dk}\,K_{jd} \\ &= \frac{1}{\sqrt{h}}\,\delta_{im}\,K_{jk}, \end{aligned} \tag{103}$$

where the Kronecker deltas δ_{im} and δ_{dk} enforce query position and channel alignment, sifting the channel sum to $d = k$. We chain the score gradient:

$$\begin{aligned}\frac{\partial L}{\partial Q_{mk}} &= \sum_{i<T}\sum_{j<T}\frac{\partial L}{\partial S_{ij}}\frac{\partial S_{ij}}{\partial Q_{mk}} \\ &= \sum_{i<T}\sum_{j<T}\frac{\partial L}{\partial S_{ij}}\,\frac{1}{\sqrt{h}}\,\delta_{im}\,K_{jk} \\ &= \frac{1}{\sqrt{h}}\sum_{j<T}\frac{\partial L}{\partial S_{mj}}\,K_{jk} \\ &= \left(\frac{1}{\sqrt{h}}\frac{\partial L}{\partial S}\,K\right)_{mk}, \end{aligned} \tag{104}$$

where the query position delta sifts the outer sum over i to m. The remaining sum over key position j represents the inner product contraction between the rows of $\frac{\partial L}{\partial S}$ and the columns of K, yielding $\frac{\partial L}{\partial Q} = \frac{1}{\sqrt{h}}\frac{\partial L}{\partial S}\,K$.

e) Local derivative $\frac{\partial S}{\partial K}$: With target score S_{ij} and probe key K_{mk} (key position m, channel k):

$$\begin{aligned}\frac{\partial S_{ij}}{\partial K_{mk}} &= \frac{\partial}{\partial K_{mk}}\left(\frac{1}{\sqrt{h}}\sum_{d<h} Q_{id}\,K_{jd}\right) \\ &= \frac{1}{\sqrt{h}}\sum_{d<h} Q_{id}\,\delta_{jm}\,\delta_{dk} \\ &= \frac{1}{\sqrt{h}}\,\delta_{jm}\,Q_{ik}, \end{aligned} \tag{105}$$

where the row delta δ_{jm} indicates key position alignment, and the channel delta sifts the sum to $d = k$. We chain the score gradient:

$$\begin{aligned}\frac{\partial L}{\partial K_{mk}} &= \sum_{i<T}\sum_{j<T}\frac{\partial L}{\partial S_{ij}}\frac{\partial S_{ij}}{\partial K_{mk}} \\ &= \sum_{i<T}\sum_{j<T}\frac{\partial L}{\partial S_{ij}}\,\frac{1}{\sqrt{h}}\,\delta_{jm}\,Q_{ik} \\ &= \frac{1}{\sqrt{h}}\sum_{i<T}\frac{\partial L}{\partial S_{im}}\,Q_{ik} \\ &= \frac{1}{\sqrt{h}}\sum_{i<T}\left(\frac{\partial L}{\partial S}^\top\right)_{mi} Q_{ik} \\ &= \left(\frac{1}{\sqrt{h}}\frac{\partial L}{\partial S}^\top Q\right)_{mk}, \end{aligned} \tag{106}$$

where the key position delta sifts the inner sum over j down to $j = m$. The remaining sum over query position i contracts the column dimension of $\frac{\partial L}{\partial S}$ with the row dimension of Q. To express this as a matrix multiplication, we transpose $\frac{\partial L}{\partial S}$ so that the summed index i sits as the column index of the left factor, which yields the matrix product $\frac{\partial L}{\partial K} = \frac{1}{\sqrt{h}}\frac{\partial L}{\partial S}^\top Q$.

f) FlashAttention-2 Simplification: Softmax Statistics Contraction: To avoid materializing the intermediate $T \times T$ matrices $\frac{\partial L}{\partial A}$ and $\frac{\partial L}{\partial S}$ in DRAM, FlashAttention-2 [3] simplifies the softmax statistics term D_i. We substitute the definition of $\frac{\partial L}{\partial A_{ik}} = \sum_{d<h}\frac{\partial L}{\partial Y_{id}}V_{kd}$ from (100) into the definition of D_i (102) to obtain:

$$\begin{aligned} D_i &= \sum_{k<T}\left(\sum_{d<h}\frac{\partial L}{\partial Y_{id}}\,V_{kd}\right)A_{ik} \\ &= \sum_{d<h}\frac{\partial L}{\partial Y_{id}}\left(\sum_{k<T}A_{ik}\,V_{kd}\right) \\ &= \sum_{d<h}\frac{\partial L}{\partial Y_{id}}\,Y_{id}, \end{aligned} \tag{107}$$

where the forward pass output Y_{id} replaces the sum over key positions k in the second line via (96). This contraction yields a key mathematical optimization: the row statistics term D_i is the row-wise dot product of the output Y and its gradient $\frac{\partial L}{\partial Y}$. Precomputing $D_i \in \mathbb{R}^T$ directly from the output tensors allows us to evaluate the score gradients $\frac{\partial L}{\partial S_{ij}}$ block-by-block in SRAM:

$$\frac{\partial L}{\partial S_{ij}} \;=\; A_{ij}\left(\sum_{d<h}\frac{\partial L}{\partial Y_{id}}\,V_{jd} - D_i\right), \tag{108}$$

eliminating the need to write the large $\frac{\partial L}{\partial A}$ matrix to DRAM.

g) FlashAttention-2 Memory-Efficient Block Tiling: Standard backpropagation materializes the $T \times T$ matrices S, A, $\frac{\partial L}{\partial A}$, and $\frac{\partial L}{\partial S}$ in DRAM, which creates a massive $O(T^2)$ memory footprint. FlashAttention-2 avoids this bottleneck by tiling the sequence dimension and executing the backward pass in SRAM in two passes:

1) **Pass 1 (dQ Computation):** We tile query positions into blocks of size B_r (indexing rows i). For each block, we:
   - load Q_i, $\frac{\partial L}{\partial Y}_i$, Y_i, and L_i (the LSE vector) from DRAM to SRAM;
   - compute the row scaling term $D_i = \sum_{d<h} \frac{\partial L}{\partial Y}_{id} Y_{id}$ in SRAM;
   - loop over key/value blocks j of size B_c: load K_j, V_j to SRAM, compute the score block $S_{ij} = Q_i K_j^\top / \sqrt{h}$, mask elements where $j > i$, apply softmax $A_{ij} = \exp(S_{ij} - L_i)$, compute the gradient slice $\frac{\partial L}{\partial S_{ij}} = A_{ij}(\frac{\partial L}{\partial Y}_i V_j^\top - D_i)$, and accumulate:
   $$\frac{\partial L}{\partial Q_i} \mathrel{+}= \frac{1}{\sqrt{h}} \frac{\partial L}{\partial S_{ij}} K_j. \tag{109}$$
   - write the fully accumulated $\frac{\partial L}{\partial Q_i}$ to DRAM once the loop over j completes.
2) **Pass 2 (dK and dV Computation):** We tile key and value positions into blocks of size B_c (indexing columns j). For each block, we:
   - load K_j, V_j to SRAM and initialize local accumulators $\frac{\partial L}{\partial K_j} = 0$, $\frac{\partial L}{\partial V_j} = 0$;
   - loop over query blocks i of size B_r: load Q_i, $\frac{\partial L}{\partial Y}_i$, Y_i, and L_i, compute D_i, reconstruct the local slices A_{ij} and $\frac{\partial L}{\partial S_{ij}}$, and accumulate:
   $$\frac{\partial L}{\partial V_j} \mathrel{+}= A_{ij}^\top \frac{\partial L}{\partial Y}_i, \tag{110}$$
   $$\frac{\partial L}{\partial K_j} \mathrel{+}= \frac{1}{\sqrt{h}} \frac{\partial L}{\partial S_{ij}}^\top Q_i. \tag{111}$$
   - write the fully accumulated $\frac{\partial L}{\partial K_j}$ and $\frac{\partial L}{\partial V_j}$ to DRAM once the loop over i completes.

Both passes are contention-free (Pass 1 parallelizes over i, while Pass 2 parallelizes over j) and execute without global atomic operations.

Result. For each head, the gradients w.r.t. Queries Q, Keys K, and Values V are:

$$\frac{\partial L}{\partial Q} = \frac{1}{\sqrt{h}} \frac{\partial L}{\partial S} K, \tag{112}$$
$$\frac{\partial L}{\partial K} = \frac{1}{\sqrt{h}} \frac{\partial L}{\partial S}^\top Q, \tag{113}$$
$$\frac{\partial L}{\partial V} = A^\top \frac{\partial L}{\partial Y}, \tag{114}$$

where the causal-masked score gradient $\frac{\partial L}{\partial S} \in \mathbb{R}^{T \times T}$ is:

$$\frac{\partial L}{\partial S_{ij}} = \begin{cases} A_{ij}\left(\frac{\partial L}{\partial A_{ij}} - D_i\right) & j \le i, \\ 0 & j > i, \end{cases} \tag{115}$$

with the intermediate term and scaling statistics:

$$\frac{\partial L}{\partial A_{ij}} = \sum_{d<h} \frac{\partial L}{\partial Y_{id}} V_{jd}, \tag{116}$$
$$D_i = \sum_{d<h} \frac{\partial L}{\partial Y_{id}} Y_{id}. \tag{117}$$

IX. Embeddings (Encoder)

The input representation encodes both token vocabulary identity and sequence position. Following the standard language modeling paradigm [7], [8], the forward pass computes the input sequence embeddings x_{bt} by adding token and position embeddings, or in index form:

$$x_{bt,c} = W_{E,\,\mathrm{tok}_{bt},\,c} + W_{P,\,t,\,c}, \tag{118}$$

where token indices satisfy $\mathrm{tok}_{bt} \in \{0, \dots, V-1\}$, token embeddings occupy $W_E \in \mathbb{R}^{V \times C}$, position embeddings occupy $W_P \in \mathbb{R}^{T \times C}$, and the channel index is $c < C$.

Derivation. Let b, t, and c denote the batch, sequence position, and channel indices. We introduce independent probe indices $k < T$ (position), $v < V$ (vocab token), and $j < C$ (channel).

a) Gradient w.r.t. position embeddings: To compute the gradient w.r.t. position embeddings, we probe $W_{P,\,k,\,j}$ (position k, channel j). Because the token lookup remains constant, we differentiate the position term using (5):

$$\frac{\partial W_{P,\,t,\,c}}{\partial W_{P,\,k,\,j}} = \delta_{tk}\,\delta_{cj}, \tag{119}$$

which yields the local derivative:

$$\frac{\partial x_{bt,c}}{\partial W_{P,\,k,\,j}} = \delta_{tk}\,\delta_{cj}. \tag{120}$$

We chain the upstream gradient $\frac{\partial L}{\partial x_{bt,c}}$ using (8):

$$\begin{aligned} \frac{\partial L}{\partial W_{P,\,k,\,j}} &= \sum_{b<B} \sum_{t<T} \sum_{c<C} \frac{\partial L}{\partial x_{bt,c}} \frac{\partial x_{bt,c}}{\partial W_{P,\,k,\,j}} \\ &= \sum_{b<B} \sum_{t<T} \sum_{c<C} \frac{\partial L}{\partial x_{bt,c}}\,\delta_{tk}\,\delta_{cj}. \end{aligned} \tag{121}$$

The Kronecker deltas act as switches that vanish for all terms except where the indices match: δ_{tk} is non-zero only when $t = k$, and δ_{cj} is non-zero only when $c = j$. These deltas sift the summation, collapsing the double sum over position t and channel c to a single term where t is replaced by k and c is replaced by j. The sum over the batch dimension b remains because the positional embeddings W_P are shared across all batches, meaning they receive gradient updates from every batch element:

$$\frac{\partial L}{\partial W_{P,\,k,\,j}} = \sum_{b<B} \frac{\partial L}{\partial x_{bk,j}}, \tag{122}$$

with free indices (k, j). Because the network shares $W_{P,t}$ across batches, the gradient reduces over the batch dimension b.

b) Gradient w.r.t. token embeddings: To compute the gradient w.r.t. token embeddings, we probe $W_{E,\,v,\,j}$ (vocab token v, channel j), which requires comparing the lookup index tok_{bt} against v. We write the embedding lookup in indicator form:

$$W_{E,\,\mathrm{tok}_{bt},\,c} \;=\; \sum_{u<V} \mathbb{1}[\mathrm{tok}_{bt} = u]\; W_{E,\,u,\,c}. \tag{123}$$

We differentiate with respect to $W_{E,\,v,\,j}$:

$$\begin{aligned}\frac{\partial W_{E,\,\mathrm{tok}_{bt},\,c}}{\partial W_{E,\,v,\,j}} &= \sum_{u<V} \mathbb{1}[\mathrm{tok}_{bt} = u]\; \frac{\partial W_{E,\,u,\,c}}{\partial W_{E,\,v,\,j}} \\ &= \sum_{u<V} \mathbb{1}[\mathrm{tok}_{bt} = u]\; \delta_{uv}\, \delta_{cj} \\ &= \mathbb{1}[\mathrm{tok}_{bt} = v]\; \delta_{cj}, \end{aligned} \tag{124}$$

which yields the local derivative:

$$\frac{\partial x_{bt,c}}{\partial W_{E,\,v,\,j}} \;=\; \mathbb{1}[\mathrm{tok}_{bt} = v]\; \delta_{cj}. \tag{125}$$

We chain the upstream gradient $\frac{\partial L}{\partial x_{bt,\,c}}$ using (8):

$$\begin{aligned}\frac{\partial L}{\partial W_{E,\,v,\,j}} &= \sum_{b<B} \sum_{t<T} \sum_{c<C} \frac{\partial L}{\partial x_{bt,c}}\, \frac{\partial x_{bt,c}}{\partial W_{E,\,v,\,j}} \\ &= \sum_{b<B} \sum_{t<T} \sum_{c<C} \frac{\partial L}{\partial x_{bt,c}}\, \mathbb{1}[\mathrm{tok}_{bt} = v]\; \delta_{cj}. \end{aligned} \tag{126}$$

The delta δ_{cj} sifts the sum over channel c to j. The indicator function $\mathbb{1}[\mathrm{tok}_{bt} = v]$ evaluates to 1 when the active token at position t in batch b equals the vocab index v, and 0 otherwise. When we chain the upstream gradient, this indicator acts as a filter on the remaining sums over batch b and position t, accumulating gradients only from the specific sequence positions where token v was present:

$$\frac{\partial L}{\partial W_{E,\,v,\,j}} \;=\; \sum_{b<B} \sum_{t<T} \frac{\partial L}{\partial x_{bt,j}}\, \mathbb{1}[\mathrm{tok}_{bt} = v]\,, \tag{127}$$

with free indices (v, j). This operation scatter-accumulates upstream gradients at matching token locations, following the $+=$ convention of §I (often implemented as an index-add or scatter-add operation in deep learning libraries).

Result. The token and positional embedding gradients are:

$$\frac{\partial L}{\partial W_{E,\,v,\,c}} \;=\; \sum_{b<B} \sum_{t<T} \frac{\partial L}{\partial x_{bt,c}}\, \mathbb{1}[\mathrm{tok}_{bt} = v]\,, \tag{128}$$

$$\frac{\partial L}{\partial W_{P,\,t,\,c}} \;=\; \sum_{b<B} \frac{\partial L}{\partial x_{bt,c}}. \tag{129}$$

The positional embedding gradient $\frac{\partial L}{\partial W_P}$ reduces the upstream gradient over the batch dimension, while the token embedding gradient $\frac{\partial L}{\partial W_E}$ accumulates contributions via scatter-add at active token locations.

X. The Full Backward Pass

We trace the complete backward pass in reverse order of forward execution. Gradients flow from the cross-entropy loss, back through the language modeling head, through the stack of N_{layers} transformer blocks, and finally accumulate in the token and position embeddings. We write out the complete, coordinate-level gradient chain in index form.

A. Top-level Gradient Flow

We define the total loss L as the mean over all batch elements and sequence positions, $L = \frac{1}{B \cdot T} \sum_{b<B} \sum_{t<T} L_{bt}$. The backward pass begins by differentiating L w.r.t. the logits $Z_{bt,v}$ at position t in batch b and vocabulary entry $v < V_p$:

$$\frac{\partial L}{\partial Z_{bt,\,v}} \;=\; \frac{1}{B \cdot T}\left(p_{bt,v} - \mathbb{1}[v = y_{bt}]\right). \tag{130}$$

a) LM Head Layer: The logits are computed from the final LayerNorm output $X_{\text{final},bt,c}$ and LM head weights $W_{\text{LM},v,c}$ as $Z_{bt,v} = \sum_{c<C} X_{\text{final},bt,c} W_{\text{LM},v,c}$. We apply the coordinate-level chain rule to propagate the logit gradient:

$$\frac{\partial L}{\partial X_{\text{final},\,bt,\,c}} = \sum_{v<V_p} \frac{\partial L}{\partial Z_{bt,\,v}}\, W_{\text{LM},\,v,\,c}, \tag{131}$$

$$\frac{\partial L}{\partial W_{\text{LM},\,v,\,c}} = \sum_{b<B} \sum_{t<T} \frac{\partial L}{\partial Z_{bt,\,v}}\, X_{\text{final},\,bt,\,c}, \tag{132}$$

b) Final LayerNorm: Let $x^{(N)} \in \mathbb{R}^{B \times T \times C}$ denote the output of the last transformer block. We normalize each row to compute $X_{\text{final},bt,c} = \gamma_{f,c} \hat{x}^{(N)}_{bt,c} + \beta_{f,c}$. We chain the final LayerNorm gradient back to the block output:

$$\frac{\partial L}{\partial \gamma_{f,\,c}} = \sum_{b<B} \sum_{t<T} \frac{\partial L}{\partial X_{\text{final},\,bt,\,c}}\, \hat{x}^{(N)}_{bt,\,c}, \tag{133}$$

$$\frac{\partial L}{\partial \beta_{f,\,c}} = \sum_{b<B} \sum_{t<T} \frac{\partial L}{\partial X_{\text{final},\,bt,\,c}}, \tag{134}$$

and the input gradient:

$$\begin{aligned}\frac{\partial L}{\partial x^{(N)}_{bt,\,c}} = \frac{1}{\sqrt{\sigma^2_{bt}}} \Bigg(& \gamma_{f,c}\, \frac{\partial L}{\partial X_{\text{final},\,bt,\,c}} \\ & - \frac{1}{C} \sum_{k<C} \gamma_{f,k}\, \frac{\partial L}{\partial X_{\text{final},\,bt,\,k}} \\ & - \frac{\hat{x}^{(N)}_{bt,\,c}}{C} \sum_{k<C} \gamma_{f,k}\, \frac{\partial L}{\partial X_{\text{final},\,bt,\,k}}\, \hat{x}^{(N)}_{bt,\,k} \Bigg), \end{aligned} \tag{135}$$

where σ^2_{bt} is the variance and $\hat{x}^{(N)}_{bt,c}$ is the normalized value of row $x^{(N)}_{bt}$ computed during the forward pass.

B. The Transformer Block Stack

For layer $l = N - 1$ down to 0, the block maps the input $x^{(l)}$ to $x^{(l+1)}$ using residual connections [9] containing multi-head self-attention [8] and a multi-layer perceptron:

$$x^{\text{mid},(l)}_{bt,\,c} \;=\; x^{(l)}_{bt,\,c} + z^{\text{attn},(l)}_{bt,\,c}, \tag{136}$$

$$x^{(l+1)}_{bt,\,c} \;=\; x^{\text{mid},(l)}_{bt,\,c} + z^{\text{mlp},(l)}_{bt,\,c}, \tag{137}$$

where $z^{\text{attn},(l)}$ and $z^{\text{mlp},(l)}$ are the attention and MLP outputs. We backpropagate the block output gradient $\frac{\partial L}{\partial x^{(l+1)}_{bt,c}}$ in two steps.

a) Step One: The MLP Sub-Layer: Due to the residual connection, the upstream gradient w.r.t. $z^{\text{mlp},(l)}$ is $\frac{\partial L}{\partial z^{\text{mlp},(l)}_{bt,\,c}} = \frac{\partial L}{\partial x^{(l+1)}_{bt,\,c}}$. The MLP sub-layer forward pass evaluates:

$$a^{(l)}_{bt,\,c} = \gamma^{(l)}_{2,c}\,\hat{x}^{\text{mid},(l)}_{bt,\,c} + \beta^{(l)}_{2,c}, \tag{138}$$

$$h^{\text{fc},(l)}_{bt,\,i} = \sum_{c<C} a^{(l)}_{bt,\,c}\,W^{\text{fc},(l)}_{i,\,c} + b^{\text{fc},(l)}_{i}, \tag{139}$$

$$h^{\text{gelu},(l)}_{bt,\,i} = \text{GELU}\left(h^{\text{fc},(l)}_{bt,\,i}\right), \tag{140}$$

$$z^{\text{mlp},(l)}_{bt,\,c} = \sum_{i<4C} h^{\text{gelu},(l)}_{bt,\,i}\,W^{\text{proj},(l)}_{c,\,i} + b^{\text{proj},(l)}_{c}, \tag{141}$$

for $i < 4C$. We differentiate in reverse order. First, for the projection layer, we contract over the output channel index $c < C$:

$$\frac{\partial L}{\partial h^{\text{gelu},(l)}_{bt,\,i}} = \sum_{c<C} \frac{\partial L}{\partial z^{\text{mlp},(l)}_{bt,\,c}}\,W^{\text{proj},(l)}_{c,\,i}, \tag{142}$$

$$\frac{\partial L}{\partial W^{\text{proj},(l)}_{c,\,i}} = \sum_{b<B}\sum_{t<T} \frac{\partial L}{\partial z^{\text{mlp},(l)}_{bt,\,c}}\,h^{\text{gelu},(l)}_{bt,\,i}, \tag{143}$$

$$\frac{\partial L}{\partial b^{\text{proj},(l)}_{c}} = \sum_{b<B}\sum_{t<T} \frac{\partial L}{\partial z^{\text{mlp},(l)}_{bt,\,c}}. \tag{144}$$

Second, the GELU derivative sifts the activation gradient pointwise:

$$\frac{\partial L}{\partial h^{\text{fc},(l)}_{bt,\,i}} = \frac{\partial L}{\partial h^{\text{gelu},(l)}_{bt,\,i}}\,\text{GELU}'\left(h^{\text{fc},(l)}_{bt,\,i}\right). \tag{145}$$

Third, the first linear layer contracts over the hidden dimension $i < 4C$:

$$\frac{\partial L}{\partial a^{(l)}_{bt,\,c}} = \sum_{i<4C} \frac{\partial L}{\partial h^{\text{fc},(l)}_{bt,\,i}}\,W^{\text{fc},(l)}_{i,\,c}, \tag{146}$$

$$\frac{\partial L}{\partial W^{\text{fc},(l)}_{i,\,c}} = \sum_{b<B}\sum_{t<T} \frac{\partial L}{\partial h^{\text{fc},(l)}_{bt,\,i}}\,a^{(l)}_{bt,\,c}, \tag{147}$$

$$\frac{\partial L}{\partial b^{\text{fc},(l)}_{i}} = \sum_{b<B}\sum_{t<T} \frac{\partial L}{\partial h^{\text{fc},(l)}_{bt,\,i}}. \tag{148}$$

Fourth, we backpropagate through LayerNorm 2 using stats $\mu^{(2)}_{bt}$ and $\sigma^{2,(2)}_{bt}$:

$$\frac{\partial L}{\partial \gamma^{(l)}_{2,c}} = \sum_{b<B}\sum_{t<T} \frac{\partial L}{\partial a^{(l)}_{bt,\,c}}\,\hat{x}^{\text{mid},(l)}_{bt,\,c}, \tag{149}$$

$$\frac{\partial L}{\partial \beta^{(l)}_{2,c}} = \sum_{b<B}\sum_{t<T} \frac{\partial L}{\partial a^{(l)}_{bt,\,c}}, \tag{150}$$

and the input gradient:

$$\begin{aligned}\frac{\partial L}{\partial x^{\text{mid},(l)}_{bt,\,c}\big|_{\text{MLP}}} = \frac{1}{\sqrt{\sigma^{2,(2)}_{bt}}}\Bigg(&\gamma^{(l)}_{2,c}\,\frac{\partial L}{\partial a^{(l)}_{bt,\,c}} \\ &- \frac{1}{C}\sum_{k<C}\gamma^{(l)}_{2,k}\,\frac{\partial L}{\partial a^{(l)}_{bt,\,k}} \\ &- \frac{\hat{x}^{\text{mid},(l)}_{bt,\,c}}{C}\sum_{k<C}\gamma^{(l)}_{2,k}\,\frac{\partial L}{\partial a^{(l)}_{bt,\,k}}\,\hat{x}^{\text{mid},(l)}_{bt,\,k}\Bigg).\end{aligned} \tag{151}$$

The total mid-state gradient accumulates both pathways:

$$\frac{\partial L}{\partial x^{\text{mid},(l)}_{bt,\,c}} = \frac{\partial L}{\partial x^{(l+1)}_{bt,\,c}} + \frac{\partial L}{\partial x^{\text{mid},(l)}_{bt,\,c}\big|_{\text{MLP}}}. \tag{152}$$

b) Step Two: The Attention Sub-Layer: Due to the residual connection, the upstream gradient w.r.t. $z^{\text{attn},(l)}$ is $\frac{\partial L}{\partial z^{\text{attn},(l)}_{bt,\,c}} = \frac{\partial L}{\partial x^{\text{mid},(l)}_{bt,\,c}}$. The attention sub-layer forward pass evaluates:

$$h^{\text{attn\_inp},(l)}_{bt,\,c} = \gamma^{(l)}_{1,c}\,\hat{x}^{(l)}_{bt,\,c} + \beta^{(l)}_{1,c}, \tag{153}$$

$$qkv^{(l)}_{bt,\,m} = \sum_{c<C} h^{\text{attn\_inp},(l)}_{bt,\,c}\,W^{\text{attn},(l)}_{m,\,c} + b^{\text{attn},(l)}_{m}, \tag{154}$$

$$z^{\text{attn},(l)}_{bt,\,c} = \sum_{k<C} Y^{\text{heads},(l)}_{bt,\,k}\,W^{\text{proj},(l)}_{c,\,k} + b^{\text{proj},(l)}_{c}, \tag{155}$$

for $m < 3C$. First, we propagate gradients through the output projection:

$$\frac{\partial L}{\partial Y^{\text{heads},(l)}_{bt,\,k}} = \sum_{c<C} \frac{\partial L}{\partial z^{\text{attn},(l)}_{bt,\,c}}\,W^{\text{proj},(l)}_{c,\,k}, \tag{156}$$

$$\frac{\partial L}{\partial W^{\text{proj},(l)}_{c,\,k}} = \sum_{b<B}\sum_{t<T} \frac{\partial L}{\partial z^{\text{attn},(l)}_{bt,\,c}}\,Y^{\text{heads},(l)}_{bt,\,k}, \tag{157}$$

$$\frac{\partial L}{\partial b^{\text{proj},(l)}_{c}} = \sum_{b<B}\sum_{t<T} \frac{\partial L}{\partial z^{\text{attn},(l)}_{bt,\,c}}. \tag{158}$$

Second, we partition the gradient w.r.t. the head activations. Let H be the head count, $h = C/H$ be the head dimension, $i < H$ be the head index, and $d < h$ be the head channel index. We extract the head output gradient:

$$\frac{\partial L}{\partial Y^{(l)}_{bt,\,i,\,d}} = \frac{\partial L}{\partial Y^{\text{heads},(l)}_{bt,\,i\cdot h+d}}. \tag{159}$$

We backpropagate through the self-attention mechanism for each head i:

$$\frac{\partial L}{\partial V^{(l)}_{bj,\,i,\,d}} = \sum_{t<T} \frac{\partial L}{\partial Y^{(l)}_{bt,\,i,\,d}}\,A^{(l)}_{b,\,i,\,t,\,j}, \tag{160}$$

$$\frac{\partial L}{\partial A^{(l)}_{b,\,i,\,t,\,j}} = \sum_{d<h} \frac{\partial L}{\partial Y^{(l)}_{bt,\,i,\,d}}\,V^{(l)}_{bj,\,i,\,d}. \tag{161}$$

The row-wise softmax statistics $D^{(l)}_{b,\,i,\,t}$ contract to:

$$D^{(l)}_{b,\,i,\,t} = \sum_{d<h} \frac{\partial L}{\partial Y^{(l)}_{bt,\,i,\,d}}\,Y^{(l)}_{bt,\,i,\,d}. \tag{162}$$

The score gradient $\frac{\partial L}{\partial S^{(l)}_{b,\,i,\,t,\,j}}$ is masked causally:

$$\frac{\partial L}{\partial S^{(l)}_{b,\,i,\,t,\,j}} \;=\; \begin{cases} A^{(l)}_{b,\,i,\,t,\,j}\left(\frac{\partial L}{\partial A^{(l)}_{b,\,i,\,t,\,j}} - D^{(l)}_{b,\,i,\,t}\right) & j \le t, \\ 0 & j > t. \end{cases} \tag{163}$$

We compute query and key gradients by contracting over the sequence dimensions:

$$\frac{\partial L}{\partial Q^{(l)}_{bt,\,i,\,d}} = \frac{1}{\sqrt{h}} \sum_{j<T} \frac{\partial L}{\partial S^{(l)}_{b,\,i,\,t,\,j}}\, K^{(l)}_{bj,\,i,\,d}, \tag{164}$$

$$\frac{\partial L}{\partial K^{(l)}_{bj,\,i,\,d}} = \frac{1}{\sqrt{h}} \sum_{t<T} \frac{\partial L}{\partial S^{(l)}_{b,\,i,\,t,\,j}}\, Q^{(l)}_{bt,\,i,\,d}. \tag{165}$$

Third, we reassemble the composite projection gradient $\frac{\partial L}{\partial qkv^{(l)}_{bt,\,m}}$:

$$\frac{\partial L}{\partial qkv^{(l)}_{bt,\,m}} \;=\; \begin{cases} \frac{\partial L}{\partial Q^{(l)}_{bt,\,i,\,d}} & m = i\cdot h + d, \\ \frac{\partial L}{\partial K^{(l)}_{bt,\,i,\,d}} & m = C + i\cdot h + d, \\ \frac{\partial L}{\partial V^{(l)}_{bt,\,i,\,d}} & m = 2C + i\cdot h + d, \end{cases} \tag{166}$$

where $i = \lfloor (m \bmod C)/h \rfloor$ and $d = m \bmod h$. Fourth, we backpropagate through the QKV projection layer parameters and inputs:

$$\frac{\partial L}{\partial h^{\text{attn\_inp},(l)}_{bt,\,c}} = \sum_{m<3C} \frac{\partial L}{\partial qkv^{(l)}_{bt,\,m}}\, W^{\text{attn},(l)}_{m,\,c}, \tag{167}$$

$$\frac{\partial L}{\partial W^{\text{attn},(l)}_{m,\,c}} = \sum_{b<B}\sum_{t<T} \frac{\partial L}{\partial qkv^{(l)}_{bt,\,m}}\, h^{\text{attn\_inp},(l)}_{bt,\,c}, \tag{168}$$

$$\frac{\partial L}{\partial b^{\text{attn},(l)}_{m}} = \sum_{b<B}\sum_{t<T} \frac{\partial L}{\partial qkv^{(l)}_{bt,\,m}}. \tag{169}$$

Fifth, we backpropagate through LayerNorm 1 using stats $\mu^{(1)}_{bt}$ and $\sigma^{2,(1)}_{bt}$:

$$\frac{\partial L}{\partial \gamma^{(l)}_{1,c}} = \sum_{b<B}\sum_{t<T} \frac{\partial L}{\partial h^{\text{attn\_inp},(l)}_{bt,\,c}}\, \hat{x}^{(l)}_{bt,\,c}, \tag{170}$$

$$\frac{\partial L}{\partial \beta^{(l)}_{1,c}} = \sum_{b<B}\sum_{t<T} \frac{\partial L}{\partial h^{\text{attn\_inp},(l)}_{bt,\,c}}, \tag{171}$$

and the input gradient through the attention pathway:

$$\begin{aligned} \frac{\partial L}{\partial x^{(l)}_{bt,\,c}\big|_{\text{attn}}} = \frac{1}{\sqrt{\sigma^{2,(1)}_{bt}}} \Bigg(& \gamma^{(l)}_{1,c}\, \frac{\partial L}{\partial h^{\text{attn\_inp},(l)}_{bt,\,c}} \\ & - \frac{1}{C}\sum_{k<C} \gamma^{(l)}_{1,k}\, \frac{\partial L}{\partial h^{\text{attn\_inp},(l)}_{bt,\,k}} \\ & - \frac{\hat{x}^{(l)}_{bt,\,c}}{C}\sum_{k<C} \gamma^{(l)}_{1,k}\, \frac{\partial L}{\partial h^{\text{attn\_inp},(l)}_{bt,\,k}}\, \hat{x}^{(l)}_{bt,\,k} \Bigg). \end{aligned} \tag{172}$$

The total block input gradient accumulates both pathways:

$$\frac{\partial L}{\partial x^{(l)}_{bt,\,c}} \;=\; \frac{\partial L}{\partial x^{\text{mid},(l)}_{bt,\,c}} + \frac{\partial L}{\partial x^{(l)}_{bt,\,c}\big|_{\text{attn}}}. \tag{173}$$

C. Parameter Accumulation and Weight Tying

At the bottom of the stack, the gradient w.r.t. the input embeddings is $\frac{\partial L}{\partial x^{(0)}_{bt,\,c}}$. The forward embedding layer adds token and position embeddings, $x^{(0)}_{bt,c} = W_{E,\text{tok}_{bt},c} + W_{P,t,c}$. Differentiating w.r.t. position embeddings $W_{P,t,c}$ reduces the gradient over the batch dimension b:

$$\frac{\partial L}{\partial W_{P,\,t,\,c}} \;=\; \sum_{b<B} \frac{\partial L}{\partial x^{(0)}_{bt,\,c}}. \tag{174}$$

Differentiating w.r.t. token embeddings $W_{E,v,c}$ scatter-accumulates using the indicator function:

$$\frac{\partial L}{\partial W_{E,\,v,\,c}} \;=\; \sum_{b<B}\sum_{t<T} \frac{\partial L}{\partial x^{(0)}_{bt,\,c}}\, \mathbb{1}[\text{tok}_{bt} = v]\,. \tag{175}$$

Because the input token embeddings W_E and LM head weights W_{LM} tie ($W_E = W_{\text{LM}}$), their gradients sum. The total tied token embedding gradient is:

$$\begin{aligned} \frac{\partial L}{\partial W_{E,\,v,\,c\,\text{tied}}} = \sum_{b<B}\sum_{t<T} \Bigg(& \frac{\partial L}{\partial x^{(0)}_{bt,\,c}}\, \mathbb{1}[\text{tok}_{bt} = v] \\ & + \frac{\partial L}{\partial Z_{bt,\,v}}\, X_{\text{final},\,bt,\,c} \Bigg), \end{aligned} \tag{176}$$

which accumulates contributions from the embedding lookup at the network base and the output projection at the network top.

AI Use Statement

In writing these derivations, we used the following AI tools: Overleaf's spell check and grammar modules to correct spelling and grammar mistakes; Cursor's Code and LaTeX plugins to help with formatting and outlining sections with LaTeX autocompletion; and Cursor's Composer 2.5 for bulk LaTeX fixes and resolving compiler errors. For the mojo code we are not using any AI tools or LSPs.

Appendix I
Gradient Checking

We use a central finite difference approximation to check the numerical accuracy of each derivation:

$$\frac{\partial L}{\partial \theta_i} \;\approx\; \frac{L(\theta + \epsilon e_i) - L(\theta - \epsilon e_i)}{2\epsilon}. \tag{177}$$

Our verification tests enforce the relative (`rtol`) and absolute (`atol`) tolerances defined in `tests/_dtypes.py`, as summarized in Table II.

TABLE II
PER-DTYPE TOLERANCES FOR PYTORCH REFERENCE COMPARISON.

Dtype	Absolute Tolerance (`atol`)	Relative Tolerance (`rtol`)
`float32`	10^{-6}	10^{-5}
`float16`	10^{-3}	10^{-2}
`bfloat16`	5×10^{-3}	2×10^{-2}

APPENDIX II
USEFUL IDENTITIES

- Derivatives of basic functions:

$$\frac{\mathrm{d}}{\mathrm{d}x}\log x \;=\; \frac{1}{x}, \qquad \frac{\mathrm{d}}{\mathrm{d}x}e^x \;=\; e^x,$$
$$\frac{\mathrm{d}}{\mathrm{d}x}\tanh x \;=\; 1-\tanh^2 x.$$

- Quotient and product rules:

$$\frac{\mathrm{d}}{\mathrm{d}x}\left(\frac{u}{v}\right) = \frac{u'v-uv'}{v^2}, \quad \frac{\mathrm{d}}{\mathrm{d}x}(uv) = u'v+uv'.$$

- Kronecker delta sifting property:

$$\sum_j \delta_{jl}\, A_j \;=\; A_l.$$

- Log-sum-exp function and its gradient:

$$\frac{\partial}{\partial x_i}\log\sum_j e^{x_j} \;=\; \frac{e^{x_i}}{\sum_j e^{x_j}} \;=\; \mathrm{softmax}(x)_i.$$

REFERENCES

[1] A. Karpathy, "llm.c: LLM training in simple, raw C/CUDA," 2024. [Online]. Available: https://github.com/karpathy/llm.c

[2] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners," OpenAI blog, vol. 1, no. 8, p. 9, 2019. [Online]. Available: https://d4mucfpruotyw.cloudfront.net/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

[3] T. Dao, "FlashAttention-2: Faster attention with better parallelism and work partitioning," *arXiv preprint arXiv:2307.08691*, 2023. [Online]. Available: https://arxiv.org/abs/2307.08691

[4] D. Hendrycks and K. Gimpel, "Gaussian Error Linear Units (GELUs)," *arXiv preprint arXiv:1606.08415*, 2016. [Online]. Available: https://arxiv.org/abs/1606.08415

[5] E. Page, "Approximations to the Cumulative Normal Function and its Inverse for Use on a Pocket Calculator," *Journal of the Royal Statistical Society. Series C (Applied Statistics)*, vol. 26, no. 1, pp. 75–76, 1977.

[6] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer Normalization," *arXiv preprint arXiv:1607.06450*, 2016. [Online]. Available: https://arxiv.org/abs/1607.06450

[7] Y. Bengio, R. Ducharme, P. Vincent, and C. Jauvin, "A Neural Probabilistic Language Model," *Journal of Machine Learning Research*, vol. 3, pp. 1137–1155, 2003.

[8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008.

[9] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition," in *IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 770–778.